Case Study Session:2 | Kabaddi Matches

Title: Analyzing Kabaddi Match Events Using Exploratory Data Analysis (EDA)

Objective: *The objective of this case study is to perform an in-depth Exploratory Data Analysis (EDA) on a dataset containing detailed event information from Kabaddi matches. The goal is to gain insights into the gameplay, identify patterns, and uncover relationships between different match events and player performances. Students will learn to clean, manipulate, and visualize data using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn. Additionally, you will explore the use of group by operations and pivot tables to perform advanced aggregations and percentage calculations.*

Dataset Details: *The dataset that will be used in the class is the events dataset, which contains event-level data from Kabaddi matches with various columns. While we will focus on the events dataset, you are encouraged to explore and apply the same functions and techniques to other datasets available in the Kabaddi database provided in the drive.*

Datasets(Open link in a new tab and download the dataset.)

- ***Unnamed: 0****: Index column*
- ***clock****: Time of the event*
- ***defender_id****: ID of the defender involved in the event*
- ***defenders****: Number of defenders involved in the event*
- ***defending_all_out_points****: Points earned for all-out defense*
- ***defending_bonus_points****: Bonus points earned in defense*
- ***defending_capture_points****: Points earned for capturing the raider*
- ***defending_points****: Total points earned in defense*
- ***defending_team_id****: Team ID of the defending team*
- ***defending_technical_points****: Technical points earned in defense*
- ***do_or_die****: Indicator of a do-or-die raid*
- ***event****: Type of event (e.g., raid, defense)*
- ***event_id****: Unique identifier for the event*
- ***event_no****: Sequential number of the event*
- ***event_text****: Description of the event*
- ***match_id****: Unique identifier for the match*

- ***player_id***: *ID of the player involved in the event*
- ***raid_all_out_points***: *Points earned for all-out raid*
- ***raid_bonus_points***: *Bonus points earned in raid*
- ***raid_points***: *Total points earned in raid*
- ***raid_technical_points***: *Technical points earned in raid*
- ***raid_touch_points***: *Points earned for touching defenders*
- ***raider_bonus***: *Bonus earned by the raider*
- ***raider_id***: *ID of the raider involved in the event*
- ***raiding_team_id***: *Team ID of the raiding team*
- ***reason***: *Reason for the event (if any)*
- ***reason_id***: *Unique identifier for the reason*
- ***review.outcome***: *Outcome of the review (if any)*
- ***review.outcome.id***: *ID of the review outcome*
- ***review.outcome.outcome_value***: *Value of the review outcome*
- ***review.reason***: *Reason for the review (if any)*
- ***review.reason.id***: *ID of the review reason*
- ***review.reason.reason_value***: *Value of the review reason*
- ***review.review_taken***: *Indicator of whether a review was taken*
- ***review.team_id***: *Team ID involved in the review*
- ***score***: *Score of the match at the time of the event*
- ***status_id***: *Status ID of the event*
- ***substituted_by***: *Player who substituted another*
- ***super_raid***: *Indicator of a super raid*
- ***super_tackle***: *Indicator of a super tackle*
- ***team_id***: *ID of the team involved in the event*

Tasks:
The below tasks are general steps taken in an EDA assignment. These steps should be followed when we analyse any dataset.

1. ***Data Loading:***
   1. *Load the dataset using different methods (e.g., CSV, Excel).*
   2. *Display the first few rows to understand the structure of the dataset.*
2. ***Basic Data Exploration:***
   1. *Basic Data Exploration:*
   2. *Generate summary statistics and visualizations to understand the distribution of key variables.*
3. ***Data Cleaning:***
   1. *Handle missing values and inconsistencies in the dataset.*
   2. *Convert data types and ensure data integrity by correcting erroneous values.*

4. ***Data Transformation:***

   1. *Create new features based on existing columns to enhance the dataset.*
   2. *Apply transformations to correct, normalize, or standardize data where necessary.*

5. ***Identifying and Handling Outliers:***

   1. *Identify outliers using visual (box plots) and statistical (IQR) methods.*
   2. *Decide whether to remove or transform outliers and justify your approach.*

6. ***Descriptive Analysis and Visualization:***

   1. *Perform univariate, bivariate, and multivariate analysis using various plots and visualizations.*
   2. *Use correlation analysis to explore relationships between different variables.*

7. ***Group By and Pivot Tables:***

   1. *Use group by operations to calculate aggregated statistics (e.g., mean, sum) based on different grouping variables.*
   2. *Create pivot tables to analyze data from different perspectives, including row-wise and column-wise percentages.*
   3. *Visualize the results using heatmaps to uncover trends and patterns.*

8. ***Advanced Pivot Table Analysis:***

   1. *Explore advanced pivot table functionalities, such as calculating row-wise percentages, column-wise percentages, and percentages of the grand total.*
   2. *Use these techniques to gain deeper insights into player and team performances.*

Tasks:
By the end of this case study, you should be able to:

- *Perform a comprehensive exploratory data analysis (EDA) on a sports dataset.*
- *Clean and transform data to prepare it for analysis.*
- *Identify and handle outliers effectively.*
- *Use group by and pivot tables to perform complex data aggregations and calculations.*
- *Visualize data to communicate insights and trends clearly.*

This case study will help you develop a strong foundation in data analysis and visualization, equipping you with the skills needed to analyze real-world datasets using Python.

```python
# Importing all the dependencies requried for the Case Study ...
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
match_df = pd.read_csv("DS_match.csv")
match_df.head(10)
```

match_number	date	start_time	result	player_id_of_the_match	player_name_of_the_match	series_id	series_name	s
Match 1	7/26/2014	20:00	U Mumba beat Jaipur Pink Panthers (45 - 31)	29.0	29.0	1	Pro Kabaddi League Season 1, 2014	
Match 10	7/31/2014	21:00	Bengal Warriors beat Dabang Delhi (42 - 40)	12.0	12.0	1	Pro Kabaddi League Season 1, 2014	
			Dabang					

```python
match_df.shape
```

```
(684, 17)
```

```python
match_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 684 entries, 0 to 683
Data columns (total 17 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Unnamed: 0               684 non-null    int64
 1   match_id                 684 non-null    int64
 2   match_number             684 non-null    object
 3   date                     684 non-null    object
 4   start_time               684 non-null    object
 5   result                   679 non-null    object
 6   player_id_of_the_match   575 non-null    float64
 7   player_name_of_the_match 575 non-null    float64
 8   series_id                684 non-null    int64
 9   series_name              684 non-null    object
 10  status                   684 non-null    object
 11  toss_winner              684 non-null    int64
 12  toss_selection           684 non-null    object
 13  venue_id                 684 non-null    int64
 14  venue_name               684 non-null    object
 15  home_team_id             251 non-null    float64
 16  home_team_name           251 non-null    object
dtypes: float64(3), int64(5), object(9)
memory usage: 91.0+ KB
```

```python
players_df = pd.read_csv('DS_players.csv')
players_df.head(10)
```

	Unnamed: 0	index	player_id	player_name	player_short_name	player_skill	player_role	player_red_card	player_yellow_card
0	0	0	15	Sanjay Shrestha	Sanjay Shrestha	NaN	NaN	False	False
1	0	0	96	Rohit Rana	Rohit Rana	NaN	NaN	False	False
2	0	0	72	Navneet Gautam	Navneet Gautam	NaN	NaN	False	False
3	0	0	52	Jasvir Singh	Jasvir Singh	NaN	NaN	False	False
4	0	0	22	Wei Yang	Wei Yang	NaN	NaN	False	False
5	0	0	144	Gokul Shitole	Gokul Shitole	NaN	NaN	False	False
6	0	0	147	Prashant Chavan	Prashant Chavan	NaN	NaN	False	False
7	0	0	32	Balbir Singh	Balbir Singh	NaN	NaN	False	False
8	0	0	44	Gangadhari Mallesh	Gangadhari Mallesh	NaN	NaN	False	False
9	0	0	87	Raju Lal Choudhary	Raju Lal Choudhary	NaN	NaN	False	False

10 rows × 31 columns

```python
players_df.shape
```

```
(16892, 31)
```

```python
players_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16892 entries, 0 to 16891
Data columns (total 31 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Unnamed: 0              16892 non-null  int64
 1   index                   16892 non-null  int64
 2   player_id               16892 non-null  int64
 3   player_name             16892 non-null  object
 4   player_short_name       1698 non-null   object
 5   player_skill            0 non-null      float64
 6   player_role             0 non-null      float64
 7   player_red_card         16892 non-null  bool
 8   player_yellow_card      16892 non-null  bool
 9   player_green_card       16892 non-null  bool
 10  player_jersey           16892 non-null  int64
 11  player_played           16892 non-null  bool
```

```
team_df = pd.read_csv('DS_team.csv',low_memory = False)
team_df.head(10)
```

	Unnamed: 0	id	match_id	name	score	short_name	squad	state_of_play	state_of_play.is_raiding_now	state_of_pla
0	0	3	1	Jaipur Pink Panthers	28	JAI	[{'id': 15, 'name': 'Sanjay Shrestha', 'short_...	NaN	False	
1	1	5	1	U Mumba	44	MUM	[{'id': 54, 'name': 'Jeeva Kumar', 'short_name...	NaN	False	
2	0	2	10	Dabang Delhi K.C.	39	DEL	[{'id': 4, 'name': 'Ehsan Zamani Fard', 'short...	NaN	False	

```
team_df.shape
```

```
(1368, 34)
```

```
team_df.shape[0]/2
```

```
684.0
```

```
team_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1368 entries, 0 to 1367
Data columns (total 34 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Unnamed: 0                              1368 non-null   int64
 1   id                                      1368 non-null   int64
 2   match_id                                1368 non-null   int64
 3   name                                    1368 non-null   object
 4   score                                   1368 non-null   int64
 5   short_name                              1368 non-null   object
 6   squad                                   1368 non-null   object
 7   state_of_play                           0 non-null      float64
 8   state_of_play.is_raiding_now            596 non-null    object
 9   state_of_play.last_raid.id              596 non-null    float64
 10  state_of_play.last_raid.result          474 non-null    object
 11  state_of_play.players                   596 non-null    object
 12  state_of_play.players_on_court          596 non-null    float64
 13  stats.all_outs                          1368 non-null   int64
 14  stats.declare                           1368 non-null   int64
 15  stats.points.all_out                    1368 non-null   int64
 16  stats.points.declare                    1368 non-null   int64
 17  stats.points.extras                     1368 non-null   int64
 18  stats.points.raid_points.raid_bonus     1368 non-null   int64
 19  stats.points.raid_points.total          1368 non-null   int64
 20  stats.points.raid_points.touch          1368 non-null   int64
 21  stats.points.tackle_points.capture      1368 non-null   int64
 22  stats.points.tackle_points.capture_bonus 1368 non-null  int64
```

```python
events_df = pd.read_csv("DS_events.csv", low_memory = False)
events_df.head(10)
```

	Column1	clock	defender_id	defenders	defending_all_out_points	defending_bonus_points	defending_capture_points
0	0	0:33	NaN	NaN	NaN	NaN	NaN
1	1	0:43	NaN	NaN	NaN	NaN	NaN
2	2	0:48	54.0	NaN	NaN	NaN	NaN
3	3	1:42	NaN	NaN	NaN	NaN	NaN
4	4	2:19	NaN	NaN	NaN	NaN	NaN
5	5	2:48	NaN	NaN	NaN	NaN	NaN
6	6	3:10	NaN	NaN	NaN	NaN	NaN
7	7	3:45	147.0	NaN	NaN	NaN	NaN
8	8	3:45	96.0	NaN	NaN	NaN	NaN
9	9	4:8	123.0	NaN	NaN	NaN	NaN

10 rows × 41 columns

```python
events_df.shape
```

```
(66568, 41)
```

```python
events_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 66568 entries, 0 to 66567
Data columns (total 41 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Column1                     66568 non-null  int64
 1   clock                       66568 non-null  object
 2   defender_id                 14055 non-null  float64
 3   defenders                   0 non-null      float64
 4   defending_all_out_points    19799 non-null  float64
 5   defending_bonus_points      19799 non-null  float64
 6   defending_capture_points    19799 non-null  float64
 7   defending_points            58315 non-null  float64
 8   defending_team_id           58315 non-null  float64
 9   defending_technical_points  19799 non-null  float64
 10  do_or_die                   20297 non-null  object
 11  event                       66568 non-null  object
 12  event_id                    66568 non-null  int64
 13  event_no                    66568 non-null  int64
 14  event_text                  66568 non-null  object
 15  match_id                    66568 non-null  int64
 16  player_id                   5441 non-null   float64
 17  raid_all_out_points         19799 non-null  float64
 18  raid_bonus_points           19799 non-null  float64
 19  raid_points                 58315 non-null  float64
 20  raid_technical_points       19799 non-null  float64
 21  raid_touch_points           19799 non-null  float64
 22  raider_bonus                498 non-null    float64
```

```
events_df['event'].unique()
```

```
array(['Successful Raid', 'Empty Raid', 'Unsuccessful Raid',
       'Substitution', 'Raider and Anti Out', 'TimeOut', 'Green Card',
       'Non Raid Technical Points', 'Yellow Card', 'Red Card'],
      dtype=object)
```

```
events_df['event'].value_counts()
```

```
event
Empty Raid                  24951
Successful Raid             20218
Unsuccessful Raid           13119
Substitution                 4733
TimeOut                      2588
Green Card                    657
Non Raid Technical Points     224
Yellow Card                    48
Raider and Anti Out            27
Red Card                        3
Name: count, dtype: int64
```

```
events_df[events_df['event'] == 'Substitution'][['clock','event','event_text','match_id','player_id']]
```

	clock	event	event_text	match_id	player_id
34	16:4	Substitution	Gangadhari Mallesh comes in for Prashant Chavan	1	147.0
45	20:0	Substitution	Balbir Singh comes in for Gangadhari Mallesh	1	44.0
58	25:1	Substitution	Gangadhari Mallesh comes in for Gokul Shitole	1	144.0
205	07:10	Substitution	Varinder Singh comes in for Vijay Malik	100	208.0
207	07:29	Substitution	Vijay Malik comes in for Varinder Singh	100	260.0
...	...	...	...	...	...
66521	21:02	Substitution	Deepak Kumar comes in for Kedar Lal	99	184.0
66528	24:03	Substitution	Bajirao Hodage comes in for Vikas Kale	99	164.0
66533	26:01	Substitution	Vineet Sharma comes in for Jang Kun Lee	99	268.0
66544	31:05	Substitution	Mahendra Ganesh Rajput comes in for Deepak Kumar	99	163.0
66560	37:22	Substitution	Deepak Kumar comes in for Mahendra Ganesh Rajput	99	184.0

4733 rows × 5 columns

```
events_df[events_df['event'] == 'Successful Raid'][['clock','event','event_text','match_id','raid_points']]
```

	clock	event	event_text	match_id	raid_points
0	0:33	Successful Raid	Jasvir Singh Successful Raid	1	1.0
7	3:45	Successful Raid	Shabeer Bappu Successful Raid	1	2.0
8	3:45	Successful Raid	Shabeer Bappu Successful Raid	1	2.0
10	4:28	Successful Raid	Anup Kumar Successful Raid	1	1.0
11	5:19	Successful Raid	Jasvir Singh Successful Raid	1	2.0
...	...	...	...	...	...
66549	32:58	Successful Raid	Pradeep Kumar raids successfully	99	1.0
66556	36:00	Successful Raid	Bhupender Singh raids successfully	99	1.0
66559	36:44	Successful Raid	Anup Kumar raids successfully	99	3.0
66562	37:36	Successful Raid	Pradeep Kumar raids successfully	99	1.0
66565	39:03	Successful Raid	Mahendra Ganesh Rajput raids successfully	99	1.0

20218 rows × 5 columns

```
events_df[events_df['event']=='Unsuccessful Raid'][['clock','event','event_text','defending_points','defender_
```

	clock	event	event_text	defending_points	defender_id
2	0:48	Unsuccessful Raid	Maninder Singh Unsuccessful Raid	1.0	54.0
3	1:42	Unsuccessful Raid	Anup Kumar Unsuccessful Raid	1.0	NaN
9	4:8	Unsuccessful Raid	Maninder Singh Unsuccessful Raid	1.0	123.0
15	7:5	Unsuccessful Raid	Anup Kumar Unsuccessful Raid	1.0	NaN
18	8:44	Unsuccessful Raid	Maninder Singh Unsuccessful Raid	1.0	146.0
...	...	...	...	...	...
66555	35:18	Unsuccessful Raid	Kedar Lal Unsuccessful Raid	1.0	NaN
66557	36:13	Unsuccessful Raid	Deepak Kumar Unsuccessful Raid	1.0	71.0
66561	37:23	Unsuccessful Raid	Kedar Lal Unsuccessful Raid	1.0	NaN
66566	39:41	Unsuccessful Raid	Bhupender Singh Unsuccessful Raid	1.0	108.0
66567	39:49	Unsuccessful Raid	Jang Kun Lee Unsuccessful Raid	1.0	71.0

13119 rows × 5 columns

```python
# Identify Missing Values
events_df.isnull().sum().sort_values(ascending = False)
```

```
reason_id                      66568
defenders                      66568
review.outcome                 66568
review.reason                  66568
review.reason.reason_value     66125
review.reason.id               66125
review.outcome.id              66125
review.outcome.outcome_value   66125
raider_bonus                   66070
reason                         65646
substituted_by                 61835
player_id                      61127
team_id                        58318
defender_id                    52513
raid_touch_points              46769
review.review_taken            46769
review.team_id                 46769
super_raid                     46769
super_tackle                   46769
raid_technical_points          46769
raid_bonus_points              46769
raid_all_out_points            46769
defending_all_out_points       46769
defending_bonus_points         46769
defending_capture_points       46769
defending_technical_points     46769
do_or_die                      46271
```

```python
# .describe # Statisitcal Analysis
events_df.describe() # default -> Numerical Columns
```

	Column1	defender_id	defenders	defending_all_out_points	defending_bonus_points	defending_capture_points	defe
count	66568.000000	14055.000000	0.0	19799.000000	19799.000000	19799.000000	!
mean	48.710326	386.817645	NaN	0.028486	0.021112	0.218950	
std	28.798869	557.652491	NaN	0.236989	0.143762	0.413545	
min	0.000000	4.000000	NaN	0.000000	0.000000	0.000000	
25%	24.000000	96.000000	NaN	0.000000	0.000000	0.000000	
50%	48.000000	179.000000	NaN	0.000000	0.000000	0.000000	
75%	73.000000	401.000000	NaN	0.000000	0.000000	0.000000	
max	122.000000	2672.000000	NaN	2.000000	1.000000	1.000000	

8 rows × 30 columns

```
events_df.describe(include = 'all') # Categorical + Numerical
```

	Column1	clock	defender_id	defenders	defending_all_out_points	defending_bonus_points	defending_capture_poir
count	66568.000000	66568	14055.000000	0.0	19799.000000	19799.000000	19799.0000
unique	NaN	3212	NaN	NaN	NaN	NaN	Na
top	NaN	19:58	NaN	NaN	NaN	NaN	Na
freq	NaN	303	NaN	NaN	NaN	NaN	Na
mean	48.710326	NaN	386.817645	NaN	0.028486	0.021112	0.2189
std	28.798869	NaN	557.652491	NaN	0.236989	0.143762	0.4135
min	0.000000	NaN	4.000000	NaN	0.000000	0.000000	0.0000
25%	24.000000	NaN	96.000000	NaN	0.000000	0.000000	0.0000
50%	48.000000	NaN	179.000000	NaN	0.000000	0.000000	0.0000
75%	73.000000	NaN	401.000000	NaN	0.000000	0.000000	0.0000
max	122.000000	NaN	2672.000000	NaN	2.000000	1.000000	1.0000

11 rows × 41 columns

```
events_df.nunique()
```

```
Column1                      123
clock                       3212
defender_id                  536
defenders                      0
defending_all_out_points       2
defending_bonus_points         2
defending_capture_points       2
defending_points               4
defending_team_id             45
defending_technical_points     3
do_or_die                      2
event                         10
event_id                      10
event_no                     123
event_text                  5413
match_id                     684
player_id                    610
raid_all_out_points            2
raid_bonus_points              2
raid_points                   12
raid_technical_points          5
raid_touch_points              6
```

```
raid_points                      12
raid_technical_points             5
raid_touch_points                 6
raider_bonus                      2
raider_id                       631
raiding_team_id                  45
reason                           20
reason_id                         0
review.outcome                    0
```

```
# Data Cleaning
# -> Fix Data Types, Removing Duplicates , Missing Values , Inconsistent Text
# Converting the Data Types
events_df.dtypes
```

```
Column1                        int64
clock                         object
defender_id                  float64
defenders                    float64
defending_all_out_points     float64
defending_bonus_points       float64
defending_capture_points     float64
defending_points             float64
defending_team_id            float64
defending_technical_points   float64
do_or_die                     object
event                         object
event_id                       int64
event_no                       int64
event_text                    object
match_id                       int64
player_id                    float64
raid_all_out_points          float64
raid_bonus_points            float64
raid_points                  float64
raid_technical_points        float64
raid_touch_points            float64
raider_bonus                 float64
raider_id                    float64
raiding_team_id              float64
```

```
events_df['do_or_die'].unique()
```

```
array([nan, False, True], dtype=object)
```

```
events_df['reason'].unique()
```

```
events_df['reason'].unique()
```

```
array([nan, 'Pulling of clothes', 'Other', 'Arguing with officials',
       'Failed to start raid within 5 seconds', 'Dangerous play',
       'Dissent', 'More than one raid', 'Improper holding',
       'Raider warned or instructed by teammates', 'Reckless foul',
       'Excessive / unnecessary violence',
       'Raiding team player crosses boundary', 'Misconduct',
       'Player leaves court during suspension of game', 'Injuring',
       'Yellow card awarded to the coach', 'Improper Holding',
       'Disturbance in play', 'Deliberate time wasting tactics',
       'Time Out violation'], dtype=object)
```

```
events_df['review.outcome.outcome_value'].unique()
```

```
array([nan, 'Unsuccessful', 'Successful'], dtype=object)
```

```
events_df['review.outcome'].unique()
```

```
array([nan])
```

```
events_df['score'].unique()
```

```
array(['[0, 1]', '[1, 2]', '[1, 3]', ..., '[52, 27]', '[52, 28]',
       '[52, 30]'], dtype=object)
```

```
events_df['super_raid'].unique()
```

```
array([nan, False, True], dtype=object)
```

```
events_df['super_tackle'].unique()
```

```
array([nan, False, True], dtype=object)
```

```
# Removing Duplicates:
print("Duplicates Before: " , events_df.duplicated().sum())
events_df = events_df.drop_duplicates()
print("Duplicates After: " , events_df.duplicated().sum())
```

```
Duplicates Before:  0
Duplicates After:  0
```

```
events_df.shape
```

```
(66568, 41)
```

```
# Dropping the fully Null Columns
events_df.dropna(axis = 1, how = 'all' , inplace = True)
events_df.shape
```

```
(66568, 37)
```

```
Column Name                    Null Value Counts
reason_id                        66568
defenders                        66568
review.outcome                   66568
review.reason                    66568
```

```
events_df.head(5)
```

	Column1	clock	defender_id	defending_all_out_points	defending_bonus_points	defending_capture_points	defending_points
0	0	0:33	NaN	NaN	NaN	NaN	0.0
1	1	0:43	NaN	NaN	NaN	NaN	0.0
2	2	0:48	54.0	NaN	NaN	NaN	1.0
3	3	1:42	NaN	NaN	NaN	NaN	1.0
4	4	2:19	NaN	NaN	NaN	NaN	0.0

5 rows × 37 columns

```
events_df['defending_all_out_points'].unique()
```

```
array([nan,  0.,  2.])
```

```
events_df['defending_bonus_points'].unique()
```

```
array([nan,  0.,  1.])
```

```
events_df['defending_capture_points'].unique()
```

```
array([nan,  0.,  1.])
```

```
events_df['defending_points'].unique()
```

```
array([ 0.,  1., nan,  2.,  3.])
```

```
events_df.columns
```

```
Index(['Column1', 'clock', 'defender_id', 'defending_all_out_points',
       'defending_bonus_points', 'defending_capture_points',
       'defending_points', 'defending_team_id', 'defending_technical_points',
       'do_or_die', 'event', 'event_id', 'event_no', 'event_text', 'match_id',
       'player_id', 'raid_all_out_points', 'raid_bonus_points', 'raid_points',
       'raid_technical_points', 'raid_touch_points', 'raider_bonus',
       'raider_id', 'raiding_team_id', 'reason', 'review.outcome.id',
       'review.outcome.outcome_value', 'review.reason.id',
       'review.reason.reason_value', 'review.review_taken', 'review.team_id',
       'score', 'status_id', 'substituted_by', 'super_raid', 'super_tackle',
       'team_id'],
      dtype='object')
```

```
events_df.columns[-1]
```

```
'team_id'
```

```
type(events_df.columns[-1])
```

```
str
```

```python
# Handling Missing values by recognizing common patterns
# Pattern - 1 : ['points'] columns -> fillna(0)

points_cols = [col for col in events_df.columns if col.endswith('points')]
points_cols
```

```
['defending_all_out_points',
 'defending_bonus_points',
 'defending_capture_points',
 'defending_points',
 'defending_technical_points',
 'raid_all_out_points',
 'raid_bonus_points',
 'raid_points',
 'raid_technical_points',
 'raid_touch_points']
```

```python
len(points_cols)
```

```
10
```

```python
# Long Hand:
point_cols = []
for col in events_df.columns:
    if col.endswith('points'):
        point_cols.append(col)

point_cols
```

```
['defending_all_out_points',
 'defending_bonus_points',
 'defending_capture_points',
 'defending_points',
 'defending_technical_points',
 'raid_all_out_points',
 'raid_bonus_points',
 'raid_points',
 'raid_technical_points',
 'raid_touch_points']
```

```python
# Identifying missing Values on 'points' cols
events_df[point_cols].isnull().sum()
```

```
defending_all_out_points      46769
defending_bonus_points        46769
defending_capture_points      46769
defending_points               8253
defending_technical_points    46769
raid_all_out_points           46769
raid_bonus_points             46769
raid_points                    8253
raid_technical_points         46769
raid_touch_points             46769
dtype: int64
```

```python
# LHS = RHS [Handling Missing Values]
events_df[point_cols] = events_df[point_cols].fillna(0)
```

```python
# After
# Identifying missing Values on 'points' cols
events_df[point_cols].isnull().sum()
```

```
defending_all_out_points      0
defending_bonus_points        0
defending_capture_points      0
defending_points              0
defending_technical_points    0
raid_all_out_points           0
raid_bonus_points             0
raid_points                   0
raid_technical_points         0
raid_touch_points             0
dtype: int64
```

```python
events_df.head(5)
```

	Column1	clock	defender_id	defending_all_out_points	defending_bonus_points	defending_capture_points	defending_points
0	0	0:33	NaN	0.0	0.0	0.0	0.0
1	1	0:43	NaN	0.0	0.0	0.0	0.0
2	2	0:48	54.0	0.0	0.0	0.0	1.0
3	3	1:42	NaN	0.0	0.0	0.0	1.0
4	4	2:19	NaN	0.0	0.0	0.0	0.0

5 rows × 37 columns